

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP
THÀNH PHỐ HỒ CHÍ MINH
Khoa Công Nghệ Thông Tin

ĐỀ THI THỰC HÀNH

Môn học phần: Phát triển giao diện ứng dụng

Lớp/Lớp học phần: DHKTPM19ATT

Ngày thi: 04/05/2026

Thời gian làm bài: 35 phút
(Không kể thời gian phát đề)

Xây dựng phần mềm My Book Library

Hình thức: được sử dụng AI để làm bài.

1. Hướng dẫn làm bài

- Sinh viên tải file mã nguồn mẫu BookLibrary_Starter.zip, giải nén và chạy: `npm install && npm run dev`
- Bài kiểm tra đã thiết kế sẵn giao diện, cấu trúc thư mục và các TODO. Sinh viên KHÔNG sửa giao diện, chỉ điền phần logic thiếu vào các vị trí có dấu TODO (Câu X) tương ứng trong code.
- Sinh viên tạo link API và thay thế link đã được ghi trong file `src/services/bookApi.js`. Nếu endpoint không hoạt động, sinh viên dùng tạm mảng dữ liệu cứng và ghi chú trong file `README.md`.
- Nộp bài: nén lại thư mục (KHÔNG kèm `node_modules`) thành `<MSSV>_<HoTen>.zip`

2. Thang điểm chi tiết

Câu	Nội dung	Điểm
1	Component, props, useRef (focus input), memo	0.5
2	Fetch API bằng axios + useEffect	1.0
3	Khởi tạo Recoil atoms (booksState, userState, filterState)	0.5
4	Custom hook useLocalStorage	1.0
5	useParams + fetch chi tiết sách	1.0
6	Lọc sách (filter + search) + useMemo tính thống kê	1.5
7	useReducer quản lý form + thêm sách (POST)	1.5
8	Xóa sách (DELETE) + cập nhật state	1.0
9	Đăng nhập/Đăng xuất + đồng bộ localStorage & Recoil	1.0
10	ProtectedRoute + điều hướng bằng React Router	1.0
	TỔNG	10.0

3. Mô tả ứng dụng

Ứng dụng là một trang web quản lý tủ sách cá nhân gồm 5 trang:

- /: Trang chủ – hiển thị thống kê tổng số sách, số sách đã đọc / đang đọc / chưa đọc.
- /books: Danh sách sách – có thanh tìm kiếm theo tên và bộ lọc theo trạng thái.

- /books/:id : Trang chi tiết một cuốn sách.
- /add : Form thêm sách mới.
- /login : Trang đăng nhập (admin/123).

Cấu trúc dữ liệu 1 cuốn sách: { id, title, author, category, status: 'read' | 'reading' | 'unread', description }

4. Đề bài

Câu 1: (0.5đ) – Component con & useRef

Mở 2 file: src/components/BookItem.jsx và src/components/SearchBox.jsx. Yêu cầu:

- Trong SearchBox.jsx: dùng useRef + useEffect để tự động focus vào ô input ngay khi component mount.
- Trong BookItem.jsx: trả lời ngắn (viết bằng comment trong file) cho 2 câu: (1) Component nhận những props nào? (2) Vì sao nên bọc bằng React.memo khi danh sách sách lớn?

Câu 2: (1.0đ) – Fetch API bằng axios

Mở src/services/bookApi.js và src/pages/BookListPage.jsx:

- Viết hàm getBooks() dùng axios.get gọi tới API_URL, trả về mảng sách.
- Trong BookListPage, dùng useEffect gọi getBooks() khi component mount, lưu kết quả vào Recoil state booksState, đồng thời hiển thị trạng thái 'Đang tải..' khi đang fetch.

Câu 3: (0.5đ) – Recoil atoms

Mở src/store/atoms.js. File đã khai báo sẵn booksState, userState, filterState nhưng chưa có giá trị default đúng. Yêu cầu:

- Đặt default cho booksState là mảng rỗng, userState là null, filterState là chuỗi 'all'.
- Import và sử dụng các atoms này tại các trang tương ứng (BookListPage, HomePage, LoginPage).

Câu 4: (1.0đ) – Custom hook useLocalStorage

Mở src/hooks/useLocalStorage.js. Viết custom hook useLocalStorage(key, initialValue) đáp ứng:

- Khi khởi tạo: đọc giá trị từ localStorage theo key; nếu chưa có thì dùng initialValue.
- Trả về [value, setValue] giống useState. Khi gọi setValue phải đồng thời ghi vào localStorage.
- Xử lý JSON.parse / JSON.stringify để lưu được cả object.

Câu 5: (1.0đ) – Route động & chi tiết sách

Mở src/pages/BookDetailPage.jsx:

- Dùng useParams để lấy id của sách từ URL.
- Viết hàm getBookById(id) trong bookApi.js và gọi trong useEffect để lấy chi tiết. Hiển thị đầy đủ title, author, category, status, description.
- Nút 'Quay lại' dùng useNavigate(-1) – đã có sẵn trong giao diện.

Câu 6: (1.5đ) – Filter, Search và useMemo

Mở src/pages/BookListPage.jsx và src/pages/HomePage.jsx:

- BookListPage: lọc sách hiển thị dựa trên filterState (all / read / reading / unread) và keyword (tìm theo title, không phân biệt hoa thường).
- HomePage: dùng useMemo để tính { total, read, reading, unread } từ danh sách sách, giúp không tính lại khi state khác thay đổi.

- Giải thích ngắn bằng comment: vì sao chỗ này dùng useMemo là phù hợp?

Câu 7: (1.5đ) – useReducer quản lý form + thêm sách

Mở src/pages/AddBookPage.jsx và bookApi.js:

- Hoàn thiện formReducer cho 3 action: SET_FIELD, SET_ERROR, RESET.
- Viết hàm addBook(book) dùng axios.post gửi sách lên server.
- handleSubmit: validate title và author không được rỗng; nếu hợp lệ thì gọi addBook, cập nhật booksState (thêm vào đầu/cuối mảng), reset form và điều hướng về /books.

Câu 8: (1.0đ) – Xóa sách

Trong BookListPage.jsx và bookApi.js:

- Viết hàm deleteBook(id) dùng axios.delete.
- handleDelete: hiện window.confirm trước khi xóa; gọi API; sau đó cập nhật Recoil booksState loại bỏ sách có id vừa xóa.

Câu 9: (1.0đ) – Đăng nhập, đăng xuất

Trong src/pages/LoginPage.jsx và src/App.jsx:

- LoginPage: nếu nhập đúng admin / 123 thì: (1) lưu { username: 'admin' } vào Recoil userState, (2) lưu vào localStorage thông qua custom hook useLocalStorage ở Câu 4, (3) navigate('/').
- App.jsx: khi userState khác null thì trên navbar ẩn link 'Đăng nhập', thay bằng 'Xin chào, {username}' và một nút 'Logout' (bấm vào thì xóa userState và xóa khỏi localStorage).
- Khi tải lại trang: nếu localStorage đã có user thì phải tự động khôi phục vào Recoil.

Câu 10: (1.0đ) – React Router & ProtectedRoute

Trong src/App.jsx:

- Tạo component ProtectedRoute({ children }) – nếu chưa đăng nhập thì <Navigate to="/login" />, ngược lại render children.
- Bọc các route /add và /books/:id bằng ProtectedRoute. Route / và /books vẫn cho xem tự do.
- Sau khi logout, nếu đang ở trang bị chặn thì phải tự động điều hướng về /login.

5. Tiêu chí chấm điểm chung

- Được dùng AI để làm bài
- Code chạy được, không có lỗi console đỏ.
- Tạo minh chứng chạy từng câu rõ ràng, có thời gian (vào lúc vừa làm xong)
- Đặt tên biến rõ ràng, tách component hợp lý.
- Xử lý trường hợp lỗi (API fail, dữ liệu trống, validate).
- Sao chép nguyên văn code giữa các SV: 0 điểm cho cả hai bài.
- Không có minh chứng không chấm điểm

--- HẾT ---